CareerAI

Complete Tech-Stack & Project Guide

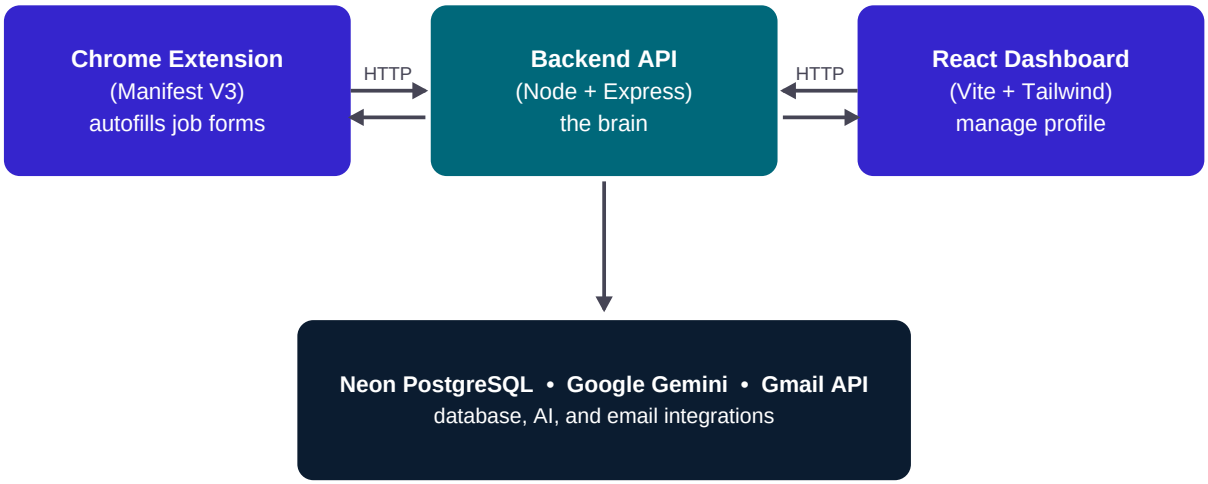
An AI-powered job-application assistant — browser extension, web dashboard, and API explained from the ground up.

Built by Soumy Dhiran

1. What is this project?

CareerAI helps you apply to jobs faster. It does three things: (1) a **Chrome extension** autofills application forms on any website, (2) an **AI assistant** drafts answers to subjective questions, and (3) a **Gmail integration** scans your inbox for interview invites and rejections. A **React dashboard** lets you manage your profile and track applications.

It is built as **three programs that talk to each other** over HTTP. Think of it like a restaurant: the **clients** (extension + website) are customers placing orders; the **server** (backend) is the kitchen that cooks them; the **database/AI/Gmail** are the pantry and specialist chefs.



All secrets live ONLY in the backend — never in the browser code.

2. The Tech Stack at a Glance

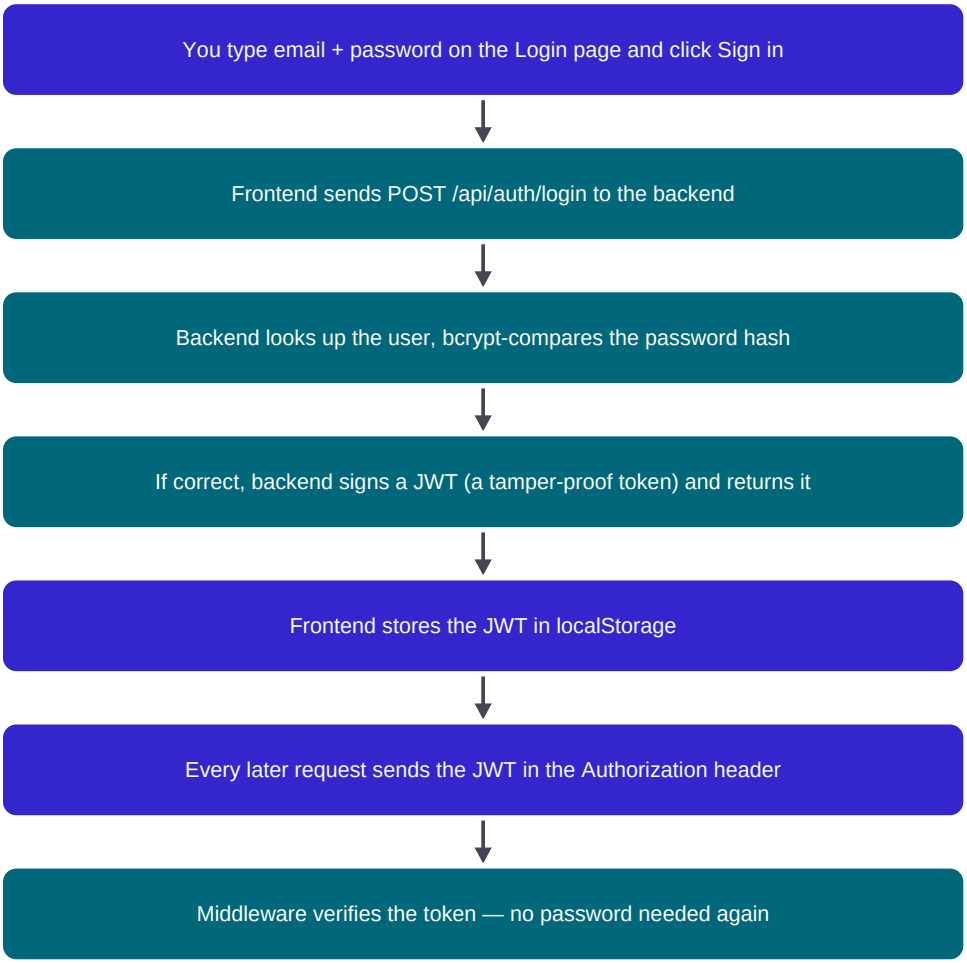
Every technology in the project and the single job it does. Each is explained in detail in Section 4.

Technology	Type	What it does (its function)	Where used
JavaScript	Language	The one language used everywhere — browser, server, extension.	Whole project
Node.js	Runtime	Runs JavaScript on the server (outside the browser).	backend/
Express.js	Web framework	Handles HTTP requests/responses and routing on the server.	backend/app.js, routes/
React 19	UI library	Builds the dashboard UI from reusable components.	frontend/src/
Vite	Build tool / dev server	Bundles the React app and serves it fast in development.	frontend/
Tailwind CSS v4	Styling	Utility classes + design tokens for the whole look.	frontend/src/index.css
React Router	Routing	Switches pages (Login/Dashboard/Profile) without reloads.	frontend/src/App.jsx
Chrome Ext (MV3)	Browser extension	Injects autofill UI into job sites; popup login.	extension/
Neon PostgreSQL	Database (SQL)	Stores users, profiles, applications on disk in the cloud.	backend/db.js
JWT	Auth tokens	Stateless 'wristband' that proves who is logged in.	routes/auth.js, middleware/
bcryptjs	Password hashing	One-way scrambles passwords so they're never stored raw.	routes/auth.js
zod	Validation	Rejects malformed requests before they hit the logic.	middleware/validate.js
express-rate-limit	Security	Caps requests to stop brute-force & runaway AI cost.	middleware/rateLimit.js
Google Gemini	LLM (AI)	Writes answers, maps fields, parses resumes.	routes/ai.js, resume.js
Gmail API + OAuth2	Integration	Read-only inbox scan for job emails, with consent.	routes/gmail.js
google-auth-library	OAuth client	Lightweight library for the Google OAuth flow.	routes/gmail.js
multer	File uploads	Receives the uploaded PDF resume in memory.	routes/resume.js
pdf-parse	PDF text	Extracts raw text from an uploaded PDF.	routes/resume.js
Render	Hosting (backend)	Runs the Node server in the cloud.	render.yaml
Vercel	Hosting (frontend)	Serves the built React site globally.	frontend/vercel.json
Git + GitHub	Version control	Tracks code history; source for deployment.	whole repo

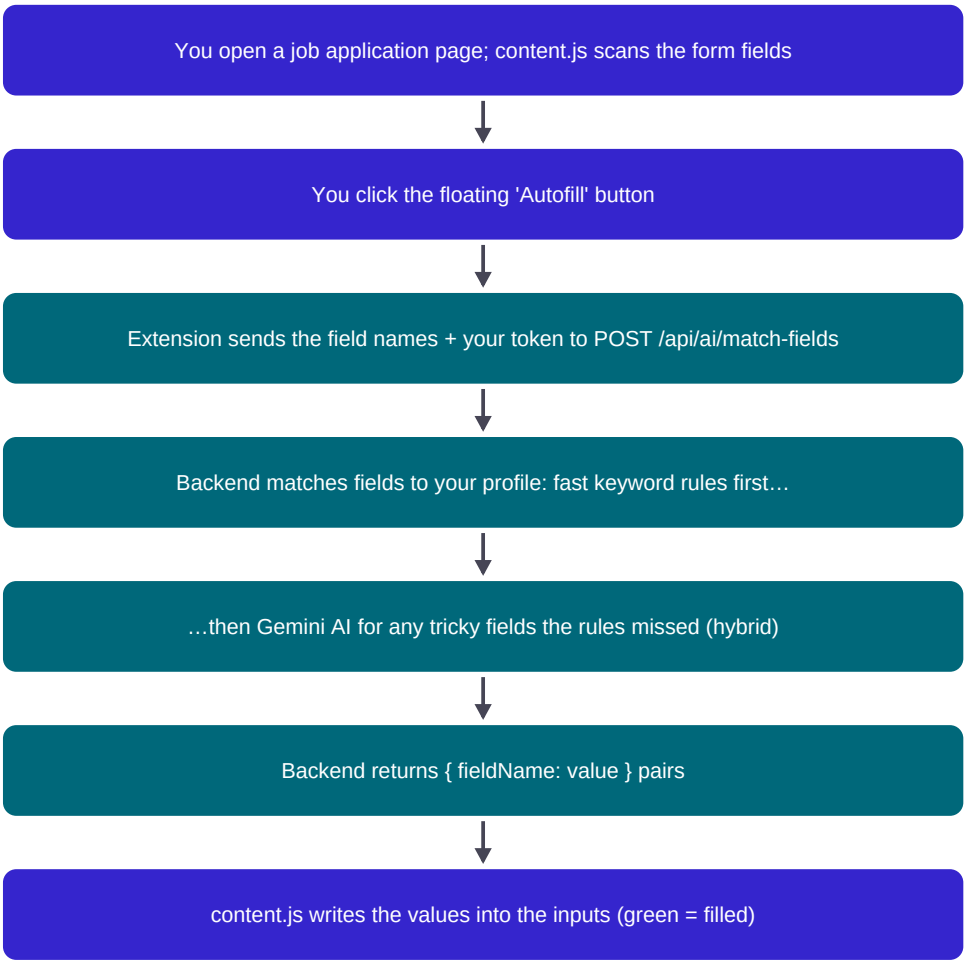
Technology	Type	What it does (its function)	Where used
node:test + supertest	Testing	Automated API tests with no network.	backend/test/

3. How it works — the key flows

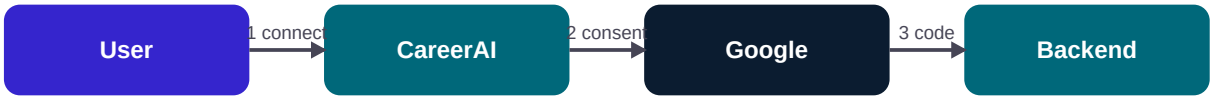
3.1 Logging in (stateless JWT auth)



3.2 Autofilling a job form (the extension)



3.3 Connecting Gmail (OAuth 2.0)



4. Backend swaps the code (+ secret) for a refresh token and stores it.

5. Later, the refresh token fetches new access tokens to read Gmail (read-only).

OAuth 2.0 = a 'valet key': limited, revocable access without sharing your password.

4. Each technology, in detail

JavaScript

What it is: The programming language of the web. The same language runs in the browser and (via Node.js) on the server.

Why we use it: Using one language across all three parts means less context-switching and shared logic.

How it's used here: All code — extension, frontend, backend — is JavaScript (with JSX for React).

Node.js

What it is: A runtime that lets JavaScript run outside the browser, directly on a computer/server.

Why we use it: It lets us build the backend server in JavaScript and use npm's huge package ecosystem.

How it's used here: Runs the Express API. Started with `node server.js`. Uses async/await for non-blocking I/O (DB, network, AI calls).

Express.js

What it is: A minimal web framework for Node that turns incoming HTTP requests into easy-to-handle 'routes'.

Why we use it: It removes boilerplate: parsing JSON bodies, routing URLs, attaching middleware.

How it's used here: app.js mounts routers like /api/auth, /api/profile, /api/applications, /api/gmail. Middleware (auth, validation, rate-limit, CORS, JSON parsing) runs in a pipeline before each route.

Key files: backend/app.js, backend/routes/*.js, backend/middleware/*.js

React 19

What it is: A library for building user interfaces out of reusable, self-contained 'components'.

Why we use it: Components + state make complex, interactive UIs manageable and predictable.

How it's used here: Pages (Login, Dashboard, Profile, ResumeUpload) are components. useState holds memory (e.g., the applications list); useEffect fetches data after render; props pass data down.

Key files: frontend/src/pages/*.jsx, components/Layout.jsx

Vite

What it is: A fast build tool and dev server for modern front-ends.

Why we use it: Instant hot-reload in dev; produces a tiny optimized bundle for production.

How it's used here: `npm run dev` serves the app locally; `npm run build` outputs static files to dist/ that Vercel hosts.

Key files: frontend/vite.config.js

Tailwind CSS v4

What it is: A utility-first CSS framework — you style with small classes like p-md, flex, text-primary.

Why we use it: Fast, consistent styling without writing custom CSS files; a design system via tokens.

How it's used here: We defined the CareerAI design tokens (colors, spacing, fonts, type scale) in @theme inside index.css, plus custom .glass-panel / .ai-gradient utilities. (Lesson learned: a custom --spacing-md token collided with max-w-md — fixed with max-w-[28rem].)

Key files: frontend/src/index.css

Chrome Extension (Manifest V3)

What it is: A small program that runs inside Chrome with three isolated 'worlds': a content script (inside web pages), a service worker (background), and a popup.

Why we use it: It's the only way to read and fill forms on third-party job sites.

How it's used here: content.js scans/fills forms and shows the floating bar; background.js (service worker) relays AI calls and shows notifications; popup.html/js is the login window; config.js centralizes the API URL. A 'sync bridge' copies the login token between the website's localStorage and the extension's chrome.storage.

Key files: extension/manifest.json, content.js, background.js, popup.*, config.js

Neon PostgreSQL

What it is: PostgreSQL is a relational SQL database (data in tables with strict columns). Neon hosts it serverlessly in the cloud.

Why we use it: We need data to persist across restarts and be queryable; SQL is reliable and well-understood.

How it's used here: db.js connects via a connection string (DATABASE_URL), creates users/profiles/applications tables, and uses parameterized queries (safe from SQL injection). The driver is lazy-loaded so the server boots fast.

Key files: backend/db.js

JWT (JSON Web Token)

What it is: A signed, tamper-proof token that encodes who you are. Like a concert wristband you flash instead of showing ID each time.

Why we use it: It makes auth 'stateless' — the server stores no session; the token itself is the proof, so it scales.

How it's used here: On login the backend signs a JWT with your id; the client sends it on every request; middleware verifies the signature. The server refuses to start without a JWT_SECRET.

Key files: backend/routes/auth.js, backend/middleware/auth.js

bcryptjs

What it is: A one-way password hashing function. You can scramble a password but never un-scramble it.

Why we use it: If the database is ever stolen, attackers see gibberish, not real passwords.

How it's used here: On register we store bcrypt.hash(password). On login we bcrypt.compare(typed, stored). The cost factor (10) controls how slow/secure the hash is.

Key files: backend/routes/auth.js

zod + express-rate-limit

What it is: zod validates the shape of incoming data; express-rate-limit caps how many requests an IP can make.

Why we use it: Validation rejects bad input with a clean 400 before it crashes deeper code. Rate-limiting stops password brute-forcing and runaway AI/Gmail cost.

How it's used here: validate(schema) middleware guards auth + applications routes; authLimiter (20/15min) and aiLimiter (15/min) guard /api/auth and /api/ai + /api/gmail.

Key files: backend/middleware/validate.js, rateLimit.js

Google Gemini (LLM)

What it is: A Large Language Model — software trained on huge text that writes human-like answers from a prompt you give it.

Why we use it: It powers the 'intelligence': drafting answers, understanding unusual form fields, parsing resumes.

How it's used here: We use a hybrid approach: cheap keyword rules first, then call Gemini only for the leftovers (controls cost + latency). Every AI feature gracefully falls back to a template/regex if no API key — the app never breaks. The free tier is enough for a demo.

Key files: backend/routes/ai.js, backend/routes/resume.js

Gmail API + OAuth 2.0

What it is: OAuth 2.0 is a 'valet key' system: the user grants your app limited, revocable access (here, read-only Gmail) without sharing their password. The Gmail API then reads messages.

Why we use it: We need to read job emails on the user's behalf, securely and with explicit consent.

How it's used here: We request the gmail.readonly scope, carry the user id through the OAuth 'state' parameter, exchange the returned code for a refresh token (stored per user), and use it to list + classify recent job emails. Uses the lightweight google-auth-library + Gmail REST (not the heavy googleapis meta-package, which hung the server).

Key files: backend/routes/gmail.js

Render + Vercel (deployment)

What it is: Render hosts the Node backend; Vercel hosts the built React frontend. Both deploy automatically from GitHub.

Why we use it: They give a public URL with HTTPS for free, and redeploy on every git push.

How it's used here: render.yaml configures the backend service + env vars; the frontend reads VITE_API_BASE to find the deployed backend; vercel.json adds SPA rewrites so client routes don't 404.

Key files: render.yaml, frontend/vercel.json, DEPLOYMENT.md

Git + GitHub

What it is: Git tracks every change to your code as 'commits' you can review or revert. GitHub stores the repo online.

Why we use it: It's the history of the project and the source Render/Vercel deploy from. Essential for any real project.

How it's used here: We init'd the repo, ignored secrets via .gitignore, committed, and pushed to GitHub. The .env file is never committed.

Key files: whole repo

5. Commands you used — and what they mean

5.1 npm & Node (run the project)

Command	What it does	Why / when
npm install	Downloads all dependencies from package.json into node_modules/.	Run once after cloning, in backend/ and frontend/.
npm start	Runs the 'start' script (node server.js) — boots the API.	Start the backend (production-style).
npm run dev	Runs Vite's dev server with hot-reload.	Start the frontend during development.
npm run build	Bundles the React app into static files in dist/.	Before deploying; Vercel runs it automatically.
npm test	Runs 'node --test' — the automated API tests.	Verify nothing broke.
node server.js	Runs the server file directly with Node.	What 'npm start' calls under the hood.
node --check file.js	Checks a file for syntax errors without running it.	Quick sanity check after editing.
node -e "..."	Runs a one-line JS snippet.	e.g. verify a package's export shape.

5.2 Git (version control)

Command	What it does	Why / when
git init -b main	Creates a new repo with 'main' as the default branch.	Once, to start tracking the project.
git status	Shows changed/staged/untracked files.	Constantly — to see what will be committed.
git add -A	Stages ALL changes for the next commit.	Before committing.
git commit -m "msg"	Saves a snapshot of staged changes with a message.	After each meaningful chunk of work.
git commit --amend	Edits the most recent commit (message or contents).	To fix the last commit before pushing.
git log --oneline	Lists commits compactly.	To review history.
git push origin main	Uploads commits to GitHub's 'main' branch.	To publish / trigger a deploy.
git push --force	Overwrites remote history with your local history.	After rewriting history (use carefully).
git rm --cached -r dir	Stops tracking files but keeps them locally.	To remove accidentally-committed files.
.gitignore	Lists files Git should never track (e.g. .env, node_modules).	Protects secrets and keeps the repo clean.

5.3 GitHub CLI (gh)

Command	What it does	Why / when
gh auth status	Shows which GitHub account is logged in.	Check you can push.
gh repo create <name> --public --source=. --push	Creates a GitHub repo from the current folder and pushes it.	Publish the project in one step.

6. Running the whole project locally

Step 1 — Backend

```
cd backend → npm install → create a .env (JWT_SECRET, DATABASE_URL, GEMINI_API_KEY, GOOGLE_* ) → npm start
```

Wait for: `Server started on port 5005` and `Neon PostgreSQL connected`.

Step 2 — Frontend

```
cd frontend → npm install → npm run dev → open http://localhost:5173
```

Step 3 — Extension

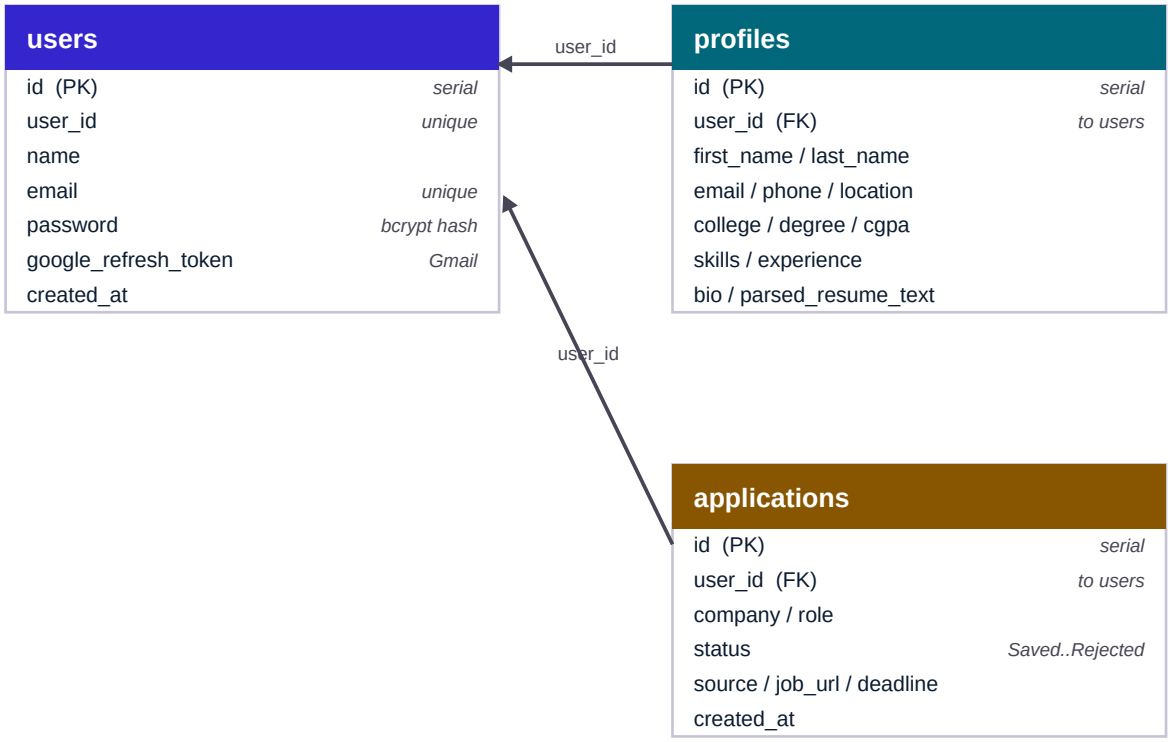
```
Chrome → chrome://extensions → enable Developer mode → Load unpacked → select the extension/ folder.
```

The .env file (backend) — never commit this

JWT_SECRET, DATABASE_URL, GEMINI_API_KEY, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI, FRONTEND_URL — these are secrets/config the backend reads at startup. They live only on your machine (and the host's env vars in production).

7. The database schema

Data lives in three PostgreSQL tables. Every profile and application is linked to a user through the **user_id** column — this link is called a **foreign key** ('this row belongs to that user'). When the dashboard loads, the backend asks: 'give me all applications where user_id = me'.



users 1-to-1 profiles, and users 1-to-many applications, linked by user_id.

Why SQL tables (not just files): they persist on disk, enforce structure, and are fast to query/filter. We always use **parameterized queries** (values sent separately from the SQL command) so a malicious input can never be executed — that defends against **SQL injection**.

8. Code walkthrough: how login works

The clearest way to understand the stack is to trace one feature through the code. Here is **login**, from `backend/routes/auth.js` + `middleware/auth.js`.

a) The server refuses to run without a secret

```
const JWT_SECRET = process.env.JWT_SECRET;
if (!JWT_SECRET) throw new Error('FATAL: JWT_SECRET is not set');
```

Tokens are signed with this secret. If it were missing and we used a default, anyone could forge logins — so we crash loudly instead.

b) The login route

```
router.post('/login', validate(loginSchema), async (req, res) => {
  const { email, password } = req.body;
  const user = await findUserByEmail(email);
  if (!user) return res.status(404).json({ error: 'Account not found' });
  const ok = await bcrypt.compare(password, user.password);
  if (!ok) return res.status(401).json({ error: 'Incorrect password' });
  const token = jwt.sign({ id: user.userId, email, name: user.name },
    JWT_SECRET, { expiresIn: '7d' });
  res.json({ token, userid: user.userId, name: user.name });
});
```

Step by step: **validate(loginSchema)** runs first (middleware) and rejects bad input with 400. **findUserByEmail** reads from PostgreSQL. **bcrypt.compare** hashes the typed password and compares to the stored hash (the raw password is never stored). On success we **sign a JWT** that expires in 7 days and return it.

c) Protecting other routes

```
const decoded = jwt.verify(authHeader.split(' ')[1], JWT_SECRET);
req.user = { id: decoded.id, name: decoded.name };
return next();
```

Every protected route runs this **authMiddleware** first. It verifies the token's signature and attaches **req.user**, so the route knows who is calling — without ever seeing the password again. The user id always comes from the verified token, never from the request body — that's why one user can't read another's data.

9. Interview prep — likely questions

Q: Walk me through your project's architecture.

A: Three parts over HTTP: a Manifest V3 Chrome extension (autofill + AI assist), a React/Vite/Tailwind dashboard, and a Node/Express API backed by PostgreSQL. Secrets live only in the backend; clients talk to it with a JWT.

Q: How does authentication work?

A: Stateless JWT. Passwords are hashed with bcrypt; on login the backend signs a token the client stores and sends on every request. Middleware verifies it — no server-side session, so it scales.

Q: How do you keep the AI cost and latency under control?

A: A hybrid approach: cheap keyword rules handle ~80% of field matching instantly and free; Gemini is called only for the leftovers. Every AI feature also falls back to a template/regex if the API is unavailable, so the app never breaks.

Q: How did you integrate Gmail securely?

A: OAuth 2.0 with the read-only gmail.readonly scope. The user grants consent on Google; the backend exchanges the code for a refresh token (stored per user) and uses it to read job emails. The user id is carried in the OAuth 'state' parameter, which also guards against CSRF.

Q: What was a hard bug you solved?

A: The server hung on startup with no error. I bisected the require() chain by timing each dependency and found the giant 'googleapis' meta-package was hanging on load. I switched to the lightweight google-auth-library + Gmail REST, and lazy-loaded other heavy modules so the server binds the port instantly.

Q: How do you prevent common web vulnerabilities?

A: Parameterized SQL queries (no injection), bcrypt-hashed passwords, JWT with a required secret, zod input validation, and express-rate-limit on auth and AI routes to stop brute-force and runaway cost. Secrets stay in .env (git-ignored).

Q: How is the extension synced with the website login?

A: A content-script 'bridge': the website and extension have separate storage, so the content script (which runs in both worlds) copies the token between them, with retries and a REQUEST_SYNC handshake to avoid a load-order race condition.

Q: How is it deployed?

A: Backend on Render, frontend on Vercel, both auto-deploying from GitHub. The frontend reads VITE_API_BASE to find the backend; a vercel.json rewrite makes client-side routing work.

10. Glossary of key terms

Term	Meaning
Client / Server	Client asks (extension, website); server answers (backend).
HTTP request/response	The order slip and the plate: method (GET/POST), URL, headers, body, status code.
API / endpoint	One action the server can do, at a URL (e.g. POST /api/auth/login).
Middleware	Code that runs on every request before the route — like security guards in a pipeline.
Async / await	Lets code wait for slow things (DB, network) without freezing everything else.
State (React)	A component's memory; changing it re-draws the screen.
DOM	The tree of elements that makes up a web page; the extension reads/edits it.
Environment variable	A config value (often secret) read from the environment, not hard-coded.
Hashing	One-way scrambling (bcrypt) — can't be reversed.
Token (JWT)	A signed proof of identity sent with each request.
OAuth scope	The exact permission granted (here: read-only Gmail).
SQL injection	An attack via malicious input; prevented by parameterized queries.
Lazy loading	Loading a heavy module only when first needed, so startup stays fast.
SPA rewrite	Server rule that serves index.html for all routes so client routing works.

This guide documents the CareerAI project end-to-end. Keep it as a reference while you learn — every concept here maps to a real file you can open and read.